

多人脸采集素材自动化清洗系统 — 概要设计文档

1. 设计概述

1.1 设计目标

本文档从系统架构、功能模块划分、数据流设计和接口规范四个层面对"多人脸采集素材自动化清洗系统"进行概要设计，为后续详细设计和编码实现提供技术蓝图。设计遵循以下原则：模块化——每个功能模块职责单一、边界清晰；可降级——核心引擎均设计有主方案和降级方案，确保系统在依赖缺失时仍可运行；前后端分离——前端为纯静态页面，后端通过 RESTful API 提供服务。

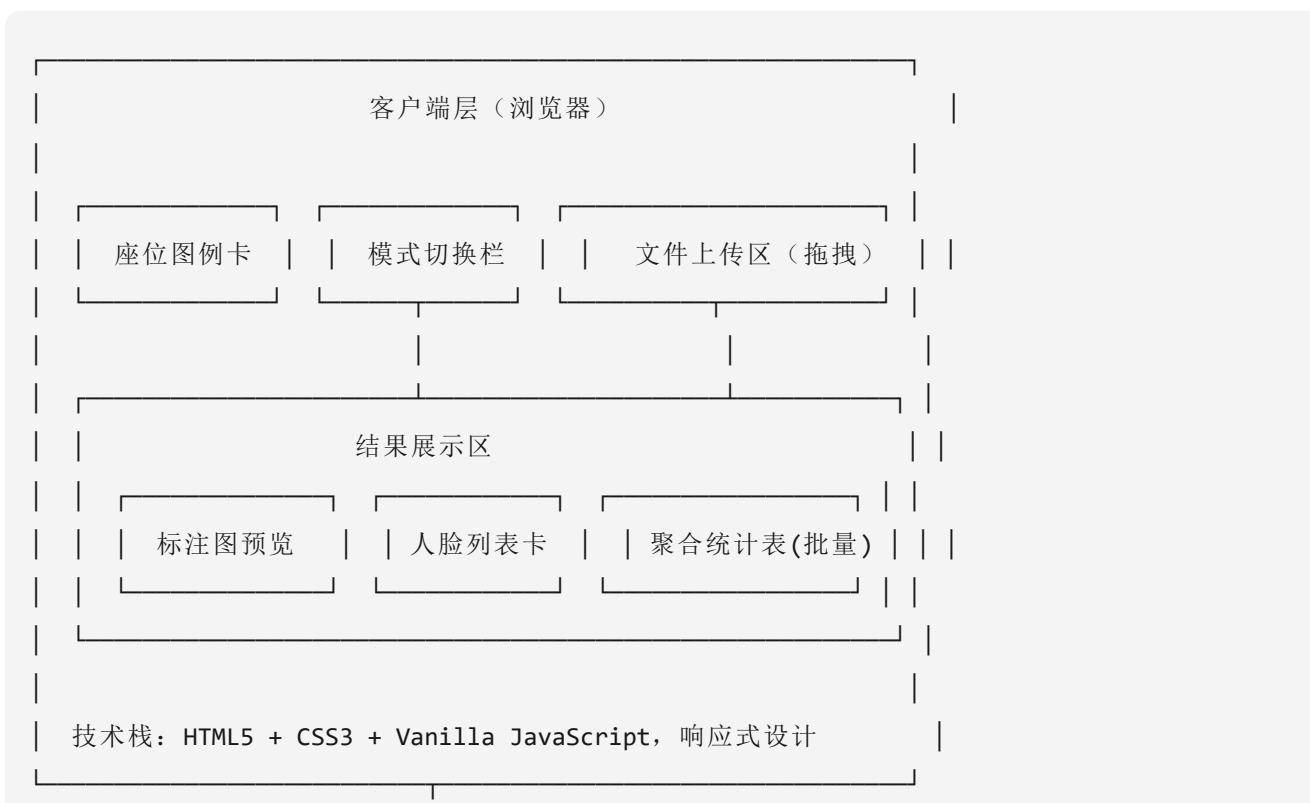
1.2 系统定位

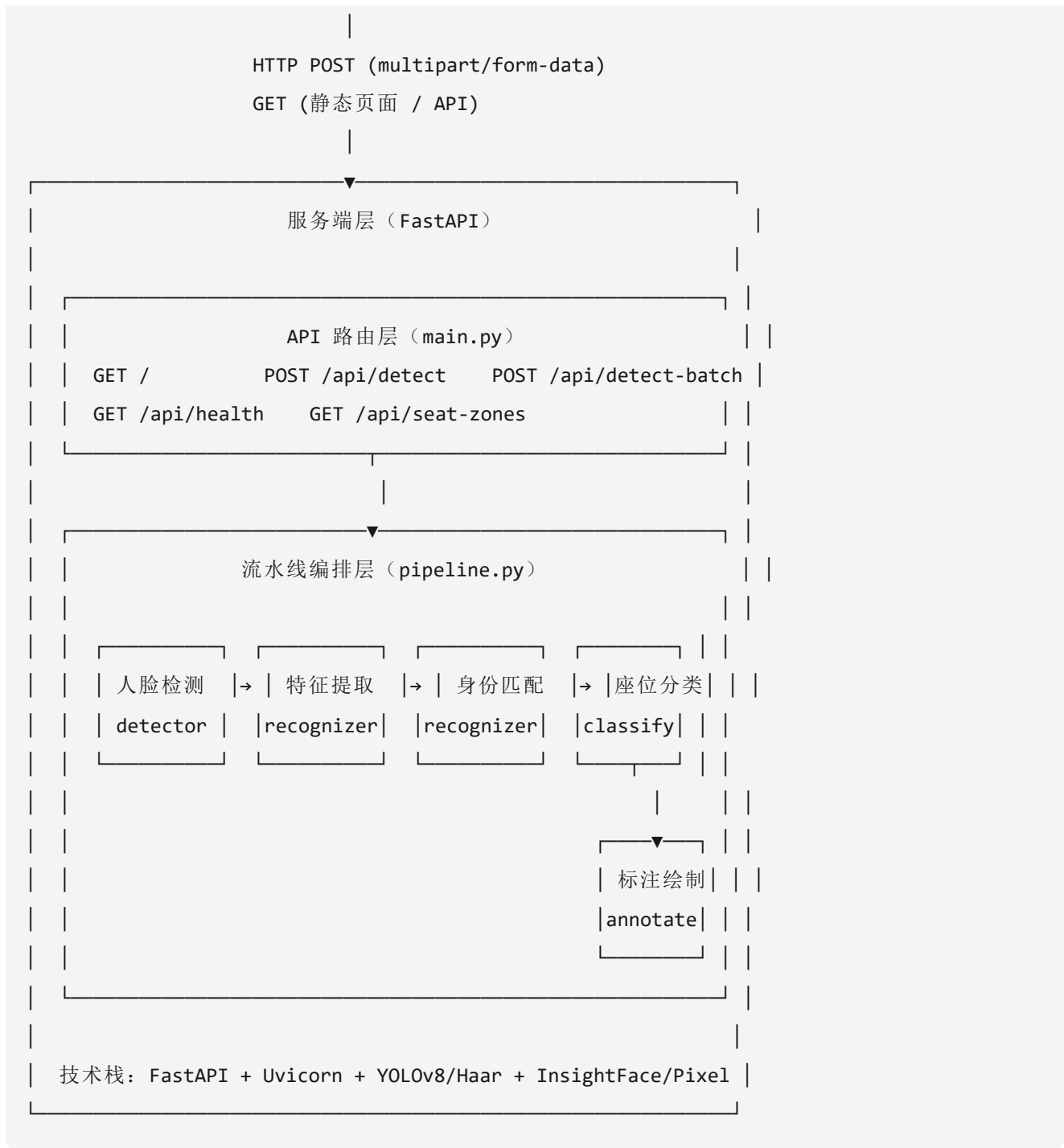
系统是一套面向车载座舱多人脸采集场景的离线数据清洗工具，以本地 Web 服务形式部署，通过浏览器访问。核心价值在于将人脸检测、身份匹配、座位分类、可视化标注和整合为端到端自动化流水线。

2. 系统架构

2.1 总体架构

系统采用经典的 B/S (Browser/Server) 架构，分为客户端层和服务端层两个层次。





2.2 技术选型说明

后端框架选择 FastAPI，原因在于其原生支持异步端点、自动 OpenAPI 文档生成、基于 Pydantic 的请求验证，且性能在 Python Web 框架中处于第一梯队。配合 Uvicorn ASGI 服务器，可高效处理文件上传等 IO 密集型任务。

人脸检测引擎以 YOLOv8-face 为主方案，基于 ultralytics 库加载。YOLOv8-face 在 WIDER FACE 等基准数据集上的精度优于传统 Haar Cascade 和 MTCNN，且支持 GPU 加速。当模型文件不存在或库未安装时，自动降级至 OpenCV Haar Cascade——后者精度较低但无需额外模型下载，适合快速验证和环境受限场景。

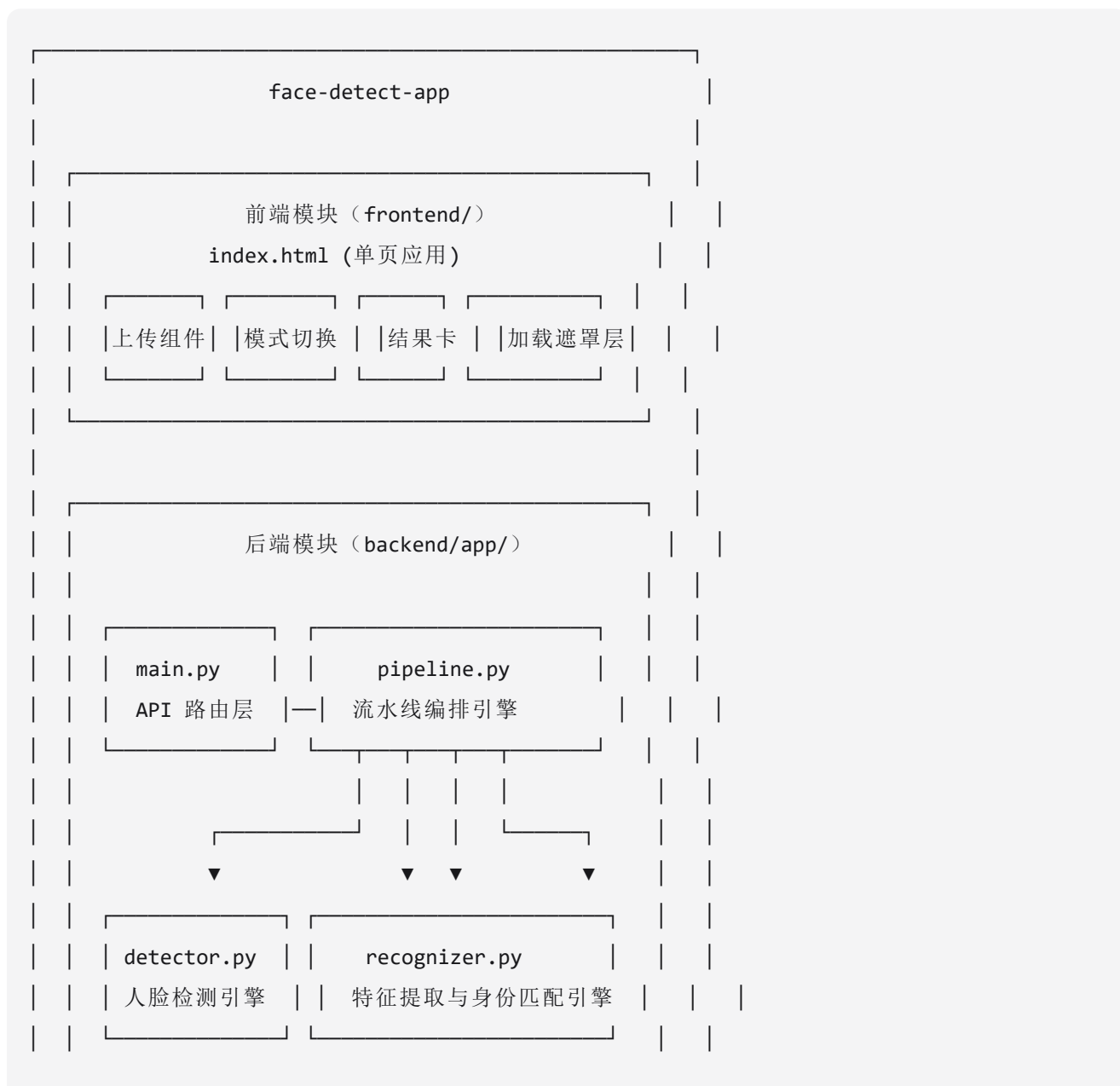
人脸特征提取设计了两级方案。主方案为 InsightFace ArcFace 512 维嵌入向量，在大规模人脸验证任务上达到 99.83% 的 LFW 精度。降级方案为基于 OpenCV 的像素特征——将人脸区域裁剪为 16x16 灰度图，进行直方图均衡化后展平为 256 维向量并 L2 归一化。降级方案精度较低但完全离线可用，无需下载任何模型文件。

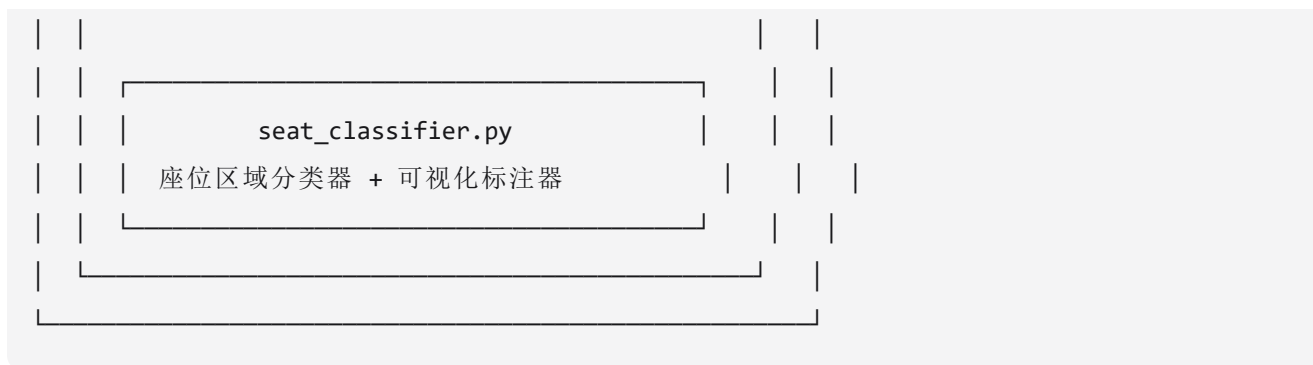
前端技术选择原生 HTML5 + CSS3 + JavaScript 单文件方案，不依赖任何构建工具或框架。这降低了部署复杂度（仅需一个静态 HTML 文件），同时保证了极低的加载速度和移动端兼容性。

3. 功能模块设计

3.1 模块总览

系统由四个核心后端模块和一个前端模块组成:





3.2 模块一：API 路由层 (main.py)

职责：定义 HTTP 端点，处理请求解析和响应封装，管理静态文件服务。

包含端点：

GET /：返回前端单页应用 (frontend/index.html)，作为静态文件服务。

GET /api/health：健康检查接口，返回 `{"status": "ok", "version": "1.0.0"}`，用于监控系统探活。

POST /api/detect：单张图像检测接口。接收 multipart/form-data 格式的图片文件（字段名 `file`），调用 `Pipeline.process_single` 处理后返回 JSON 结果。对无效输入返回 HTTP 400，对内部错误返回 HTTP 500。

POST /api/detect-batch：批量检测接口。接收多个图片文件（字段名 `files`），调用 `Pipeline.process_batch` 处理后返回 JSON。

GET /api/seat-zones：座位区域配置查询接口，返回当前生效的五个座位区域的坐标范围和中文标签。

设计要点：路由层不包含任何业务逻辑，仅负责请求/响应转换和错误处理。文件类型校验在路由层完成，仅接受 JPEG 和 PNG 格式。

3.3 模块二：流水线编排引擎 (pipeline.py)

职责：协调四个核心引擎按正确顺序执行，管理批量处理中的全局状态。

单图处理流程：接收解码后的 OpenCV 图像数组，依次执行人脸检测 → 特征提取 → 身份匹配 → 座位分类 → 可视化标注，最终返回包含结构化数据和标注图像的完整结果字典。

批量处理流程：首先对所有图片逐张执行检测和特征提取，将全部人脸信息收集到一个全局列表中。然后调用跨图身份匹配，为所有人脸统一分配 `person_id`。接着按图片维度拆分结果，对每张图片独立执行座位分类和标注。最后计算聚合统计（每人出现次数和主要座位），组装完整响应。

设计要点：Pipeline 模块持有 `FaceDetector`、`FaceRecognizer`、`SeatClassifier`、`FaceAnnotator` 四个引擎的单例引用，在应用启动时初始化。批量处理中的“先全局匹配、再逐图分类”策略确保了跨图 `person_id` 的一致性。

3.4 模块三：人脸检测引擎 (detector.py)

职责：在输入图像中定位所有人脸，返回边界框和置信度。

核心类 FaceDetector：初始化时尝试加载 YOLOv8-face 模型。如果模型文件存在且 ultralytics 库可用，则使用 YOLO 引擎；否则加载 OpenCV 内置的 Haar Cascade 分类器 (haarcascade_frontalface_default.xml)。

detect(image) 方法：YOLO 引擎对输入图像执行推理，过滤置信度低于阈值（默认 0.3）的检测结果，输出边界框列表。Haar Cascade 引擎将图像转灰度后执行多尺度检测，使用检测面积与图像面积之比作为伪置信度。两种引擎的输出格式统一为 `{"bbox": [x1, y1, x2, y2], "conf": float}`，并按 bbox 的 x1 坐标从左至右排序。

设计要点：引擎选择逻辑在 `init` 中完成，运行时不可切换。这简化了实现复杂度，同时通过 `try-except` 保证了降级路径的可靠性。

3.5 模块四：特征提取与身份匹配引擎 (recognizer.py)

职责：从裁剪后的人脸图像中提取特征向量，并通过余弦相似度实现跨图片的身份匹配。

extract_embedding(face_crop) 方法：将人脸区域裁剪为 ROI，缩放至 16x16，转灰度，应用直方图均衡化，展平为 256 维向量，L2 归一化。输出单位特征向量。

match_faces_across_images(all_faces) 方法：维护一个 `person_db` 字典，键为 `person_id`（正整数），值为该人员的特征向量。遍历所有人脸，计算当前人脸特征与 `person_db` 中每个已知身份的余弦相似度。若最高相似度超过阈值（默认 0.5），则分配对应的 `person_id`；否则创建新身份，`person_id` 递增。

设计要点：当前实现使用像素特征作为唯一方案（代码中保留了 InsightFace 的依赖声明但未实际集成）。`person_db` 的生命周期限定在单次批量请求内，请求结束后随 Pipeline 调用自动清除。

3.6 模块五：座位分类与标注引擎 (seat_classifier.py)

职责：根据人脸边界框中心坐标进行座位区域分类，以及在图像上绘制可视化标注。

SeatClassifier 类：计算每个 bbox 的中心点归一化坐标 (`center_x / width, center_y / height`)，按照预定义的五个矩形区域规则判定座位归属。区域参数存储为类属性字典，支持外部查询。

FaceAnnotator 类：将 OpenCV 图像 (BGR) 转换为 PIL 图像 (RGB)，使用 `ImageDraw` 绘制彩色矩形框和中文文字标签。中文渲染依赖系统安装字体（优先 `simhei.ttf` → `msyh.ttc` → `simsun.ttc`，路径为 `C:/Windows/Fonts/`）。标注完成后将 PIL 图像转换回 OpenCV 格式，再编码为 JPEG Base64。

设计要点：将分类和标注合并在同一模块中，是因为两者共享座位区域的颜色和标签配置。标注器选择 PIL 而非 OpenCV 的 `putText`，是因为 OpenCV 不支持直接渲染中文字符。

4. 数据流设计

4.1 单张检测数据流

用户上传图片



```
API: POST /api/detect  
解析 multipart 文件
```



```
cv2.imdecode()  
图像解码
```

原始图像字节流 → `numpy.ndarray` (BGR)



```
FaceDetector  
.detect(image)  
人脸检测
```

输入: `numpy.ndarray`

输出: `List[{bbox, conf}]`

引擎: YOLOv8-face 或 Haar Cascade



```
FaceRecognizer  
.extract_embedding  
特征提取
```

对每个检测到的人脸:

裁剪 ROI → 16x16 灰度 → 256维向量

输出: 每个 face 增加 `embedding` 字段

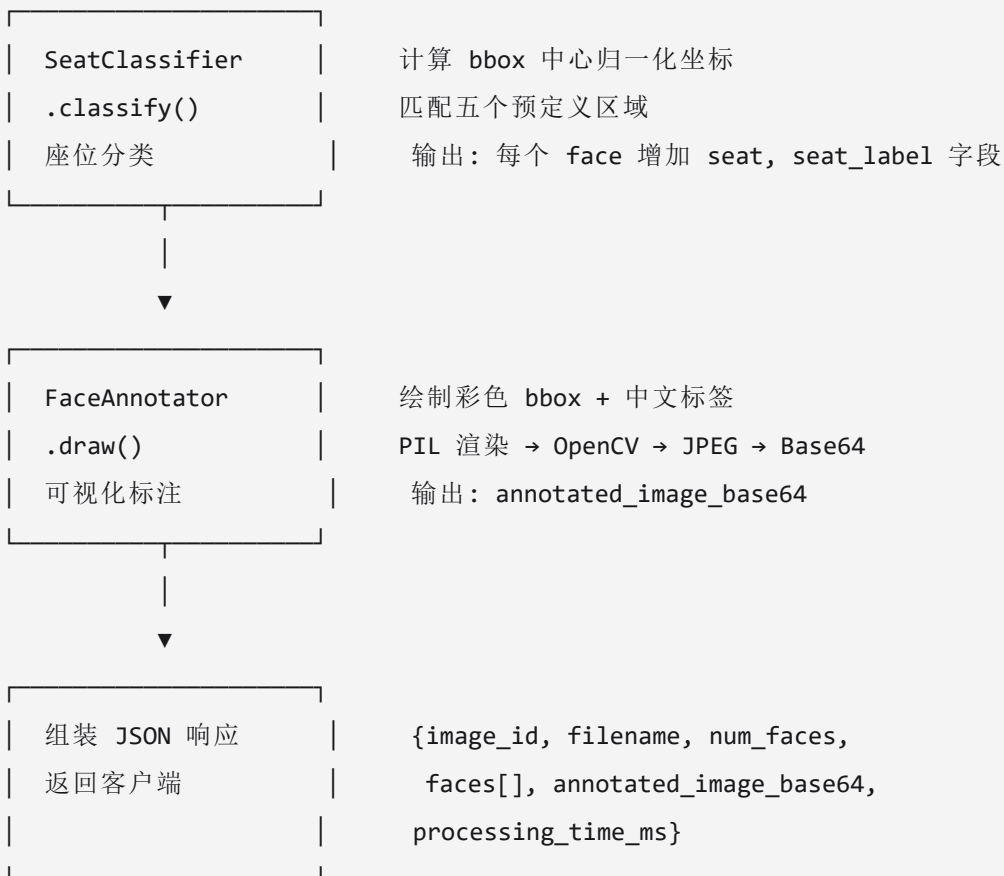


```
FaceRecognizer  
.match_faces  
身份匹配
```

与 `person_db` 中的已知身份比对

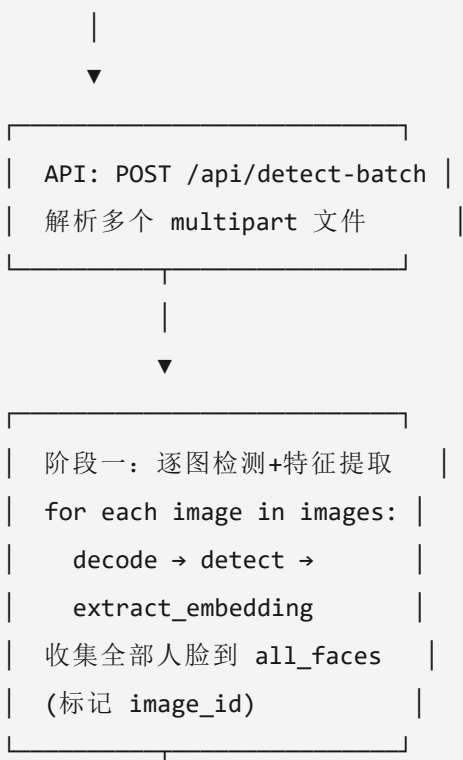
输出: 每个 face 增加 `person_id` 字段





4.2 批量处理数据流

用户上传 N 张图片



|



| 阶段二：全局身份匹配 |

| match_faces_across_images |

| 遍历 all_faces，统一分配 |

| person_id |

| person_db 全局共享 |

|



| 阶段三：逐图分类+标注 |

| for each image: |

| 按 image_id 筛选人脸 |

| classify → draw |

| 生成 per_image_result |

|



| |

| 按 person_id 分组 |

| 计算 appearance_count |

| 确定 primary_seat |

| |

| [] |

|



| 组装 JSON 响应 |

| {total_images, total_faces |

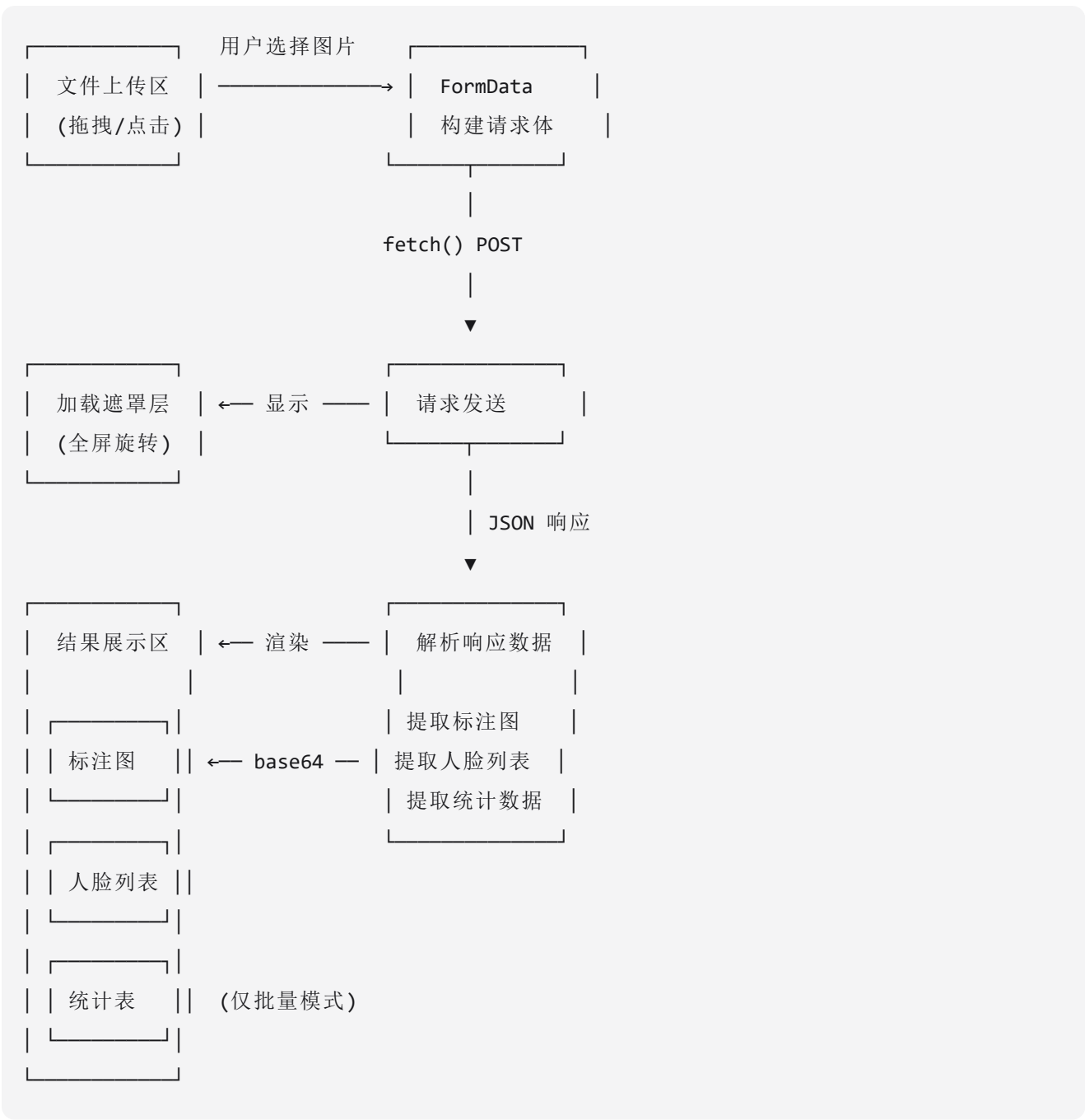
| processing_time_ms, |

| results[, |

| results[] |

| [] |

4.3 前端交互数据流



5. 接口设计

5.1 API 接口规范

所有 API 遵循 RESTful 风格，请求和响应均使用 JSON 格式（文件上传除外）。错误响应统一格式为 `{"detail": "错误描述信息"}`。

GET /api/health

响应 200:

```
{
  "status": "ok",
  "version": "1.0.0"
}
```

GET /api/seat-zones

响应 200:

```
{
  "zones": {
    "master_driver": {
      "x_range": [0.60, 1.00],
      "y_range": [0.30, 0.55],
      "label": ".."
    },
    "co_driver": {
      "x_range": [0.00, 0.40],
      "y_range": [0.30, 0.55],
      "label": ".."
    },
    "posterior_left": {
      "x_range": [0.25, 0.45],
      "y_range": [0.25, 0.45],
      "label": "...
    },
    "posterior_right": {
      "x_range": [0.45, 0.65],
      "y_range": [0.25, 0.45],
      "label": "...
    },
    "after_the_middle": {
      "x_range": [0.65, 0.90],
      "y_range": [0.25, 0.50],
      "label": "...
    }
  }
}
```

POST /api/detect

请求: multipart/form-data, 字段 file = 图片文件

响应 200:

```
{
  "image_id": 0,
  "filename": "car_01.jpg",
  "num_faces": 3,
  "faces": [
    {
      "bbox": [120, 80, 200, 170],
      "conf": 0.92,
      "seat": "master_driver",
      "seat_label": "主驾",
      "person_id": 1
    }
  ],
  "annotated_image_base64": "data:image/jpeg;base64,/9j/4AAQ...",
  "processing_time_ms": 2340.5
}
```

错误 400: {"detail": "请上传有效的图片文件"}

错误 500: {"detail": "检测处理过程中发生错误: ..."}

POST /api/detect-batch

请求: multipart/form-data, 字段 files = 多个图片文件

响应 200:

```
{
  "total_images": 3,
  "total_faces": 8,
  "processing_time_ms": 8500.0,
  "results": [
    {
      "image_id": 0,
      "filename": "img1.jpg",
      "num_faces": 3,
      "faces": [...],
      "annotated_image_base64": "..."
    }
  ]
}
```

```

],
    /* aggregate_stats ... */
    {
        "person_i
        "appearan
        "primary_seat": "master_driver",
        "primary_seat_label": "主驾"
    }
]
}

```

5.2 内部模块接口

```

# detector.py
class FaceDetector:
    def __init__(self): ...          # 自动选择 YOLOv8-face 或 Haar Cascade
    def detect(self, image: np.ndarray) -> List[dict]:
        # 返回: [{"bbox": [x1,y1,x2,y2], "conf": float}, ...]

# recognizer.py
class FaceRecognizer:
    def __init__(self): ...          # 初始化特征提取器和 person_db
    def extract_embedding(self, face_crop: np.ndarray) -> np.ndarray:
        # 返回: 256维单位特征向量
    def match_faces_across_images(self, all_faces: List[dict]) -> List[dict]:
        # 输入: 包含 embedding 字段的人脸列表
        # 输出: 增加 person_id 字段后返回

# seat_classifier.py
class SeatClassifier:
    def classify(self, bbox, image_w, image_h) -> dict:
        # 返回: {"seat": "master_driver", "seat_label": "主驾"}

class FaceAnnotator:
    def draw(self, image: np.ndarray, faces: List[dict]) -> np.ndarray:
        # 返回: 标注后的图像

# pipeline.py
class Pipeline:

```

```
def __init__(self): ...          # 初始化四个引擎单例
def process_single(self, image: np.ndarray, filename: str) -> dict:
    # 返回：单图完整结果
def process_batch(self, images: List[tuple]) -> dict:
    # 输入：[(image_array, filename), ...]
    # 返回：批量完整结果（含 results）
```

6. 错误处理设计

6.1 错误分类

系统将可能出现的错误分为三类：

输入错误 (HTTP 400)：用户上传的文件不是有效图片（cv2.imdecode 返回 None）、文件格式不在 JPEG/PNG 白名单内、上传文件数量为零。

处理错误 (HTTP 500)：检测引擎运行时异常、特征提取失败、标注绘制异常。这类错误被 try-except 捕获后返回友好的错误消息，不会导致服务进程终止。

系统错误：端口占用、依赖库缺失等启动阶段错误。通过降级机制和启动日志提示处理。

6.2 降级策略

启动时：

尝试加载 YOLOv8-face → 成功 → 使用 YOLO 引擎

|
失败
|



加载 Haar Cascade → 使用 Haar 引擎（日志警告）

运行时（特征提取）：

尝试加载 InsightFace → 成功 → 使用 ArcFace 512维

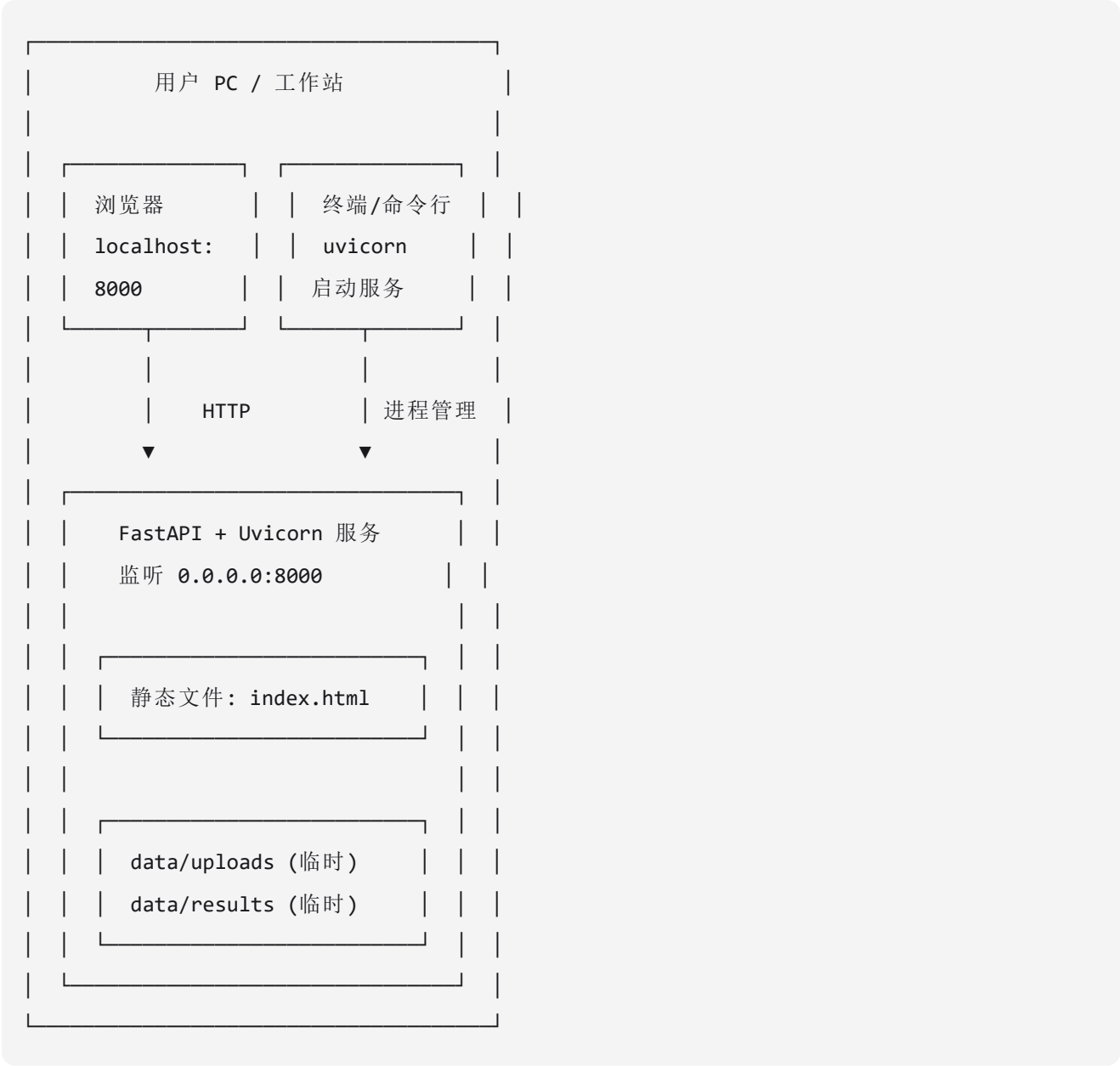
|
失败
|



使用像素特征 256维 → （日志警告）

7. 部署架构

7.1 本地部署方案



7.2 启动命令

```
cd face-detect-app/backend
pip install -r requirements.txt
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

启动后通过浏览器访问 `http://localhost:8000` 即可使用。

8. 设计约束与未来演进

8.1 当前设计约束

座位分类基于固定的坐标比例规则，适用于摄像头安装位置和视角相对固定的场景。当摄像头视角发生显著变化时，需要调整区域参数。当前实现为单进程同步处理，大批量上传时存在性能瓶颈。person_db 仅在当前请求生命周期内有效，不同批次的请求之间不共享身份信息。

8.2 未来演进方向

近期可引入异步任务队列（Celery + Redis）提升批量处理的并发能力，启用 InsightFace ArcFace 引擎提升身份匹配精度，增加 COCO/VOC 格式导出功能。中期可增加视频输入支持（自动抽帧处理）、WebSocket 实时进度推送、座位区域的前端可视化校准工具。远期可演进为多用户协作平台，支持标注结果的人工审核、修正和版本管理。